

BENJAMIN LIU

U.S. Citizen

Boston, Massachusetts

☎ 339-240-2265 ✉ benliu0001@gmail.com 🐙 github.com/benliu0001 🔗 linkedin.com/in/benliu0001

Education

University of Toronto

September 2021 - April 2026

Honors Bachelor of Science, Computer Science

Toronto, Canada

- Relevant Courses: **Computer Vision, Deep Learning, Software Engineering, Databases, Operating Systems, Algorithms, Data Structures, Multivariable Calculus, Probability and Statistics**

Technical Skills

Languages: Python, Java, C/C++, SQL, GraphQL, Hack (PHP), Kotlin

Developer Tools: Bash, Linux, Git, Agile, GCP, Docker, AWS, CMake

Technologies/Frameworks: Numpy, OpenCV, PyTorch, Pandas, Pyplot, Scipy, Transformers

Experience

Meta (Facebook)

May 2025 – August 2025

Software Engineering Intern

Bellevue, Washington

- Developed a new call-to-action (CTA) UI component for Facebook Reels ads with embedded product images, increasing ad click-through rates at billions of daily impressions scale.
- Implemented backend services in Hack (PHP) and GraphQL to handle ad eligibility, image delivery, and integration into Meta's ads serving infrastructure.
- Built high-performance UI components in Kotlin using Litho, improving rendering efficiency.
- Designed and analyzed large-scale A/B experiments using exposure-tagged data, leveraging logging and monitoring to identify traffic loss issues and ensure system reliability.

Projects

Deep Learning Side Channel Attack | *Python, PyTorch, Numpy, Scipy*

January 2024 – August 2024

- Developed LSTM, Resnet, and Transformer models to perform a deep-learning side channel attack on an STM32 microcontroller running TinyAES.
- Assumed profiling attack paradigm to collect training data using a microcontroller and oscilloscope to capture power traces of plaintext-ciphertext pairs.
- Researched various methods to extract features from power traces and optimize the models to improve the attack's success rate.
- Recruited, organized, and led a team of 6 other students to work on the project, and managed the project's timeline.

Key-Value Store (Database) | *C++, Cmake*

December 2024

- Developed a highly performant Log-Structured Merge (LSM) Tree system to manage storage across memory and disk, including seamless integration of SSTable flushing and merging mechanisms.
- Implemented an AVL tree-based Memtable for in-memory storage, optimizing for worst-case lookup time complexity.
- Built a hash table-based buffer pool with a clock eviction strategy to cache pages in memory, reducing I/O overhead.
- Implemented custom merging algorithms for SSTables, balancing memory use and processing efficiency, and conducted throughput experiments for write, read, and scan operations.

Image Generation Knowledge Distillation | *Python, PyTorch, Numpy*

January 2024 – April 2024

- Developed a model distillation technique, reducing the size of a large pre-trained VAE by 30x while maintaining comparable performance for text-to-image generation.
- Designed and implemented a 7-layer convolutional neural network to mimic both the encoder and decoder of a VAE, achieving efficient latent space encoding and image reconstruction.
- Trained models on datasets including CIFAR-10, Harvard Flower, CC12M, and Deep Fashion-MultiModal to ensure generalization across diverse image sets.
- Demonstrated strong model generalization on unseen and cross-dataset data, optimizing performance through MSE loss, dropout, and weight decay.

MIT Embedded Security CTF | *C, Python*

February 2020 – August 2020

- In a team of 4, designed and implemented a secure firmware update system for an IoT device using C and Python.
- Based on MITRE eCTF competition, assumed man in the middle paradigm to attack other teams by exploiting vulnerabilities in their firmware update systems using python scripts, earning 3rd place in the competition.
- Utilized an HMAC to authenticate users, AES, RSA, and SHA to encrypt firmware, and a Stream Cipher for key generation.
- Received intensive instruction from leading researchers on embedded software, computer architecture, memory management, assembly, cryptography, security, interface analysis, and bit manipulation.